The EBVCube Project

An Onboarding Manual

From NetCDF and EBV cubes, through the ebvcube R package, to creating and visualising your own

Written by

Paul Miegel

Geography, Leipzig University · iDiv (German Centre for Integrative Biodiversity Research, Halle–Jena–Leipzig)

The ebvcube package is distributed via CRAN, GitHub and the b-cubed r-universe.

How to use this manual

This manual introduces the EBVCube project from the ground up. It is written so that a new user or contributor can get from *“what is a netCDF?”* to *“how do I create and share an EBV cube of my own?”* without having to piece the project together from scattered sources.

Read it in order the first time. The chapters build on each other: the file format first, then the EBV-specific structure, then the tools you need to actually open the data, then the `ebvcube` package itself.

It doubles as a reference. Each chapter also stands on its own, so once you are productive you can jump straight to the function catalogue (`Chapter 5`) or the plotting recipes (`Chapter 6`).

A note on the canonical source. The most up-to-date, runnable reference is the documentation for the package, maintained alongside the source at github.com/EBVcube/ebvcube and editable from the repo. This manual summarises and frames that work; when a code example here and the online docs ever disagree, trust the online docs, because they are re-rendered against the live package. This manual reflects the ebvcube 0.5.3 version; run packageVersion("ebvcube") to check the version you have installed.

Conventions used here

`monospace` marks code, file names, function names and paths.

Green boxes like this one are notes and tips. Amber boxes are warnings or gotchas.

Code blocks are R unless a `bash` comment says otherwise. Output lines are prefixed with `#>`, following the package docs.

Contents

How to use this manual 2

1. Introduction 4
2. NetCDF fundamentals 5
3. EBV cubes: structure, hierarchy and metadata 7
4. Tools and environment 10
5. The ebvcube package 12
6. Visualisation 15
7. Creating your own EBV cube 18
8. FAQ and troubleshooting 20

Appendix A. Glossary 21
Appendix B. Key links and resources 21
Appendix C. Known limitations and contributing 21

1. Introduction

The **EBV Data Portal** and the `ebvcube` R package are two halves of one idea: make biodiversity data **findable, comparable and reusable** across studies, taxa and regions. The portal publishes datasets; the package lets people open, analyse, visualise and even create them in R. Both are developed at **iDiv** within the GEO BON framework of **Essential Biodiversity Variables (EBVs)**.

All of the data sits in a particular flavour of **netCDF** file — the "EBV cube" format. The whole stack, then, is: a self-describing scientific file format (netCDF), a biodiversity-specific structure layered on top of it (the EBV cube), a portal that hosts the files, and an R package that speaks the format fluently. Three kinds of resources sit alongside the code: this manual together with presentation slides, which explain the concepts and the workflow, and the online Quarto documentation, which is the runnable, always-current function reference in the ebvcube repository on GitHub.

1.1 Who this is for, and what you should already know

You do not need any prior experience with netCDF, the CF or ACDD conventions, or biodiversity modelling — those are introduced from scratch in Chapters 2 and 3. It will help a lot if you are reasonably comfortable with:

- **Base R** — variables, vectors, data frames and functions.
- **The idea of geospatial raster data** — a grid of cells with coordinates, a resolution, and a coordinate reference system (CRS). The packages `terra` and `sf` come up constantly.

If R is brand new to you, Section 4.1 is a self-contained quick-start you can follow step by step; if you already know R, skip it.

1.2 The resource map

Everything connected to the project, in one place. Bookmark these:

Resource	Where	What it is
EBV Data Portal	portal.geobon.org	Public catalogue of all published EBV datasets (netCDF + JSON + DOI).
ebvcube (package)	github.com/EBVcube/ebvcube	The R package source. Also where the documentation is findable.
EBVCube-format	github.com/EBVcube/EBVCube-format	The formal definition of the EBV cube netCDF structure.
EBVCubeVisualizer	github.com/EBVcube/EBVCubeVisualizer	QGIS plugin for exploring EBV cubes visually (see 4.3).
Portal How-To	portal.geobon.org/howto/site/	"how to ebv-portal" — metadata, structure, upload workflow.

2. NetCDF fundamentals

Before the EBV-specific parts make sense, it pays to understand the container they live in. This chapter is the conceptual map for the file format; Chapter 3 then layers the EBV rules on top.

2.1 What it is and why it exists

NetCDF (Network Common Data Form), developed by Unidata/UCAR, is a file format for **array-oriented scientific data** — the kind of multi-dimensional numeric grids you get in climate science, oceanography, remote sensing and, here, biodiversity. Its defining property is that it is **self-describing**: the file carries not just the numbers but also what they mean — their dimensions, units, coordinate systems and provenance — inside the same file.

That is exactly why EBV data uses it. A biodiversity metric varies across space, over time, and across many species at once; storing that as a heap of separate GeoTIFFs quickly becomes unmanageable and loses the metadata that ties them together. One netCDF holds the whole thing, described.

2.2 The data model

A netCDF file is built from four kinds of components. Internalise these four words and the rest follows:

Component	What it is
Dimensions	Named axes with a length — e.g. lon = 360, lat = 180, time = 12. They define the shape of the data.
Variables	The actual data arrays, each defined over a set of dimensions — e.g. a cube variable over [lon, lat, time, entity], or a 1-D coordinate variable lon over [lon].
Attributes	Key–value metadata attached either to a variable (e.g. units = “individuals”) or to the whole file (“global” attributes like title, creator, license).
Groups	Folder-like containers that nest variables and attributes into a hierarchy (netCDF-4 only). The EBV format uses these for scenarios and metrics.

A useful mental picture: *dimensions* are the rulers, *variables* are the data measured against those rulers, *attributes* are the sticky notes describing everything, and *groups* are the folders that organise it.

2.3 NetCDF-4 is HDF5 underneath

There are two on-disk generations. **NetCDF classic** (the older model) is flat — no groups. **NetCDF-4** is built on top of HDF5, a general hierarchical data format, and it adds groups, compression, chunking and larger data types. EBV cubes are netCDF-4 files, which is why the package depends on the HDF5 libraries (`rhdf5`) and why tools that only understand classic netCDF sometimes choke on them.

2.4 The conventions: CF and ACDD

A bare netCDF can be self-describing in any idiosyncratic way its author chose. To make files **interoperable**, the community agreed on conventions — controlled vocabularies for the attribute names and structure:

- **CF (Climate and Forecast) Conventions, v1.8** — cfconventions.org. Defines how spatial/temporal coordinates, units and cell methods are encoded, so that any CF-aware tool can place the data on a map and a timeline.
- **ACDD (Attribute Convention for Data Discovery), v1.3** — wiki.esipfed.org/Attribute_Convention_for_Data_Discovery_1-3. Defines the global attributes (title, summary, creator, keywords, license…) that let data portals index and search files.

An EBV cube is **both** a CF file and an ACDD file, plus a small set of extra EBV-specific structural rules.

2.5 Peeking inside a netCDF without R

A quick way to look inside a file before you ever touch R, covered again in Chapter 4:

- **Panoply** (NASA) — a free GUI viewer; drag a file in and browse its groups, variables and attributes, and draw quick maps.
- **EBVCubeVisualizer (QGIS plugin)** — browse a cube's groups, metadata and individual slices interactively inside QGIS, with no code.

3. EBV cubes: structure, hierarchy and metadata

This chapter is the one to come back to whenever a file does not behave the way you expected. It explains what an EBV *is*, why the community settled on netCDF, and what the internal structure of an EBV cube actually looks like.

3.1 What is an EBV?

Essential Biodiversity Variables (EBVs) are a framework defined by GEO BON for monitoring biodiversity change consistently. An EBV is a measurement that sits *between* raw observations and the high-level indicators used by policy — the common currency that lets different studies be compared.

GEO BON organises EBVs into six canonical classes:

- Genetic composition
- Species populations
- Species traits
- Community composition
- Ecosystem structure
- Ecosystem functioning

> **Six classes vs. what the portal shows**
>
> The portal's filter list reflects which classes currently have published datasets, and some appear under slightly different names (e.g. "Ecosystem function"). It also lists **Ecosystem services** as a class of its own, alongside the original six. Finer tags like "Species distribution" or "Ecosystem disturbance" are subcategories. The GEO BON definitions page is the reference when in doubt.

3.2 The four dimensions

Every data cube inside an EBV netCDF has the same four dimensions, in this order:

Dimension	Meaning
lon	Longitude (or projected x-coordinate).
lat	Latitude (or projected y-coordinate).
time	Time steps — annual, decadal, scenario years, irregular etc.
entity	The biological unit — a species, a group of species, an ecosystem type, etc.

The `entity` dimension is what makes EBV cubes distinctive. It lets a single file describe, say, 200 bird species on the same grid and timeline, instead of forcing the provider to publish 200 separate rasters.

> **A transpose gotcha you will hit**
>
> The file stores cubes in `[lon, lat, time, entity]` order (longitude first). But when `ebv_read(type = "a")` hands the data to R, it follows R's row–column convention: dimension 1 is **latitude** (rows), dimension 2 is **longitude** (columns). So `data[i, j, k]` is `data[lat, lon, time]`.

3.3 Scenarios, metrics and the group hierarchy

A single file can hold many cubes, organised into a two-level group hierarchy:

```
root/
├─ scenario_1/
│   ├─ metric_1/
│   │   └─ ebv_cube  [lon, lat, time, entity]
│   └─ metric_2/
│       └─ ebv_cube  [lon, lat, time, entity]
└─ scenario_2/
    ├─ metric_1/
    │   └─ ebv_cube  [lon, lat, time, entity]
    └─ metric_2/
        └─ ebv_cube  [lon, lat, time, entity]
```

- **Scenarios** are the outer group — e.g. the same metric under several SSP/RCP pathways. They are **optional**: a file of observed data has none.
- **Metrics** are the inner group and are **always** present — e.g. "relative change in species richness." A file may have any number.
- **EBV_cube** is the actual 4-D data array. Every cube in a file shares the same four dimensions.

> **Datacube paths — the string you will use constantly**
>
> A **datacube path** is the slash-separated route to a cube: `scenario_1/metric_2/ebv_cube`, or for files without scenarios simply `metric_1/ebv_cube`. `ebv_datacubepaths()` lists them. **Always discover paths from the file** rather than hard-coding them.

3.4 Metadata: the S4 properties object

ebvcube parses a file's metadata into an S4 object with these slots, which you reach with the `@` operator:

Slot	What it describes
@general	Title, abstract, EBV class, creator, license, DOI, entity names, taxonomy.
@spatial	CRS, bounding box (extent), resolution, dimensions.
@temporal	Start, end, step size, the vector of dates, calendar.
@metric	Name, description and units of the selected metric.
@scenario	Name and description of the selected scenario (empty if the file has none).
@ebv_cube	Variable-level metadata: units, fill value, valid range, data type.

The `@general`, `@spatial` and `@temporal` slots are filled whether or not you pass a datacube path; the per-cube slots (`@metric`, `@scenario`, `@ebv_cube`) need a path because different cubes in the same file can carry different units or fill values.

3.5 The EBV Data Portal

Every officially published EBV dataset lives on the portal, where each has:

- a stable, integer dataset ID;
- a JSON metadata record;
- a downloadable netCDF;
- a citation and a DOI.

> **IDs are not a tidy 1…n range**
>
> The portal lists on the order of ~50 datasets, but a valid ID can be larger than that (e.g. id 64), because numbers are never reused or compacted. Always look an ID up on the portal rather than assuming a range. A few datasets used as running examples in the package docs: **id 1** Local bird diversity (cSAR/BES-SIM), **id 2** Forest loss 2000–2020, **id 6** Local terrestrial diversity (PREDICTS, has scenarios), **id 30** Global trends in biodiversity (BES-SIM cSAR-iDiv), **id 64** Global trends in ecosystem services (GLOBIO-ES).

One-sentence mental model to carry forward: *an EBV netCDF is a CF-compliant netCDF holding one or more 4-D cubes* `[lon, lat, time, entity]`*, optionally nested under a scenario layer, with rich metadata describing what each cube represents.*

4. Tools and environment

Three things let you actually touch EBV data: R/RStudio (the workhorse), Panoply (a fast first look), and QGIS with the EBVCubeVisualizer plugin (visual exploration inside a GIS).

4.1 RStudio quick-start (skip if you already use R)

R is the language; RStudio is the IDE most people use to write it. Install **R first, then RStudio**:

1. Install R (≥ 4.2.0) from cran.r-project.org. Check the version in any R console with `R.version.string`.
2. Install RStudio Desktop (free) from posit.co/download/rstudio-desktop.
3. Open RStudio. The four panes are: the **console** (bottom-left, runs code now), the **source editor** (top-left, where you write and save scripts), **environment/history** (top-right), and **files/plots/packages/help** (bottom-right).

Packages extend R. Install one with `install.packages("name")` and load it for the session with `library(name)`. You install once, but `library()` every session.

> **Use an RStudio Project — it prevents the #1 beginner confusion**
>
> Create a Project (File › New Project) for your work. A project fixes R's **working directory**, which matters because the package examples cache downloads into a relative `data/ebv/` folder. If you run code from a different directory each time, R looks in a different place and re-downloads files. The docs' setup line `dir.create("data/ebv", recursive = TRUE, showWarnings = FALSE)` assumes a consistent working directory — a Project gives you exactly that.

Two functions you will lean on: `?ebv_read` opens a function's help page, and `View(object)` opens a navigable viewer — great for exploring a properties object slot by slot.

4.2 Panoply — a fast first look

Panoply is a free NASA viewer for netCDF and HDF5 files. Drag a `.nc` file in and it shows the full group hierarchy, dimensions and attributes side by side, and will draw a quick map of any slice. It is the fastest way to confirm "did the file come out as expected?" — indispensable while you are learning the format and after you create your own cube (Chapter 7).

> **Install gotcha**
>
> Panoply needs Java. On Windows, uninstalling the stock Java and installing Temurin JRE 17 (Eclipse Adoptium) is often required before Panoply will launch properly. If it refuses to start, that is the first thing to check.

4.3 QGIS and the EBVCube-Visualizer plugin

For exploring EBV cubes inside a full GIS, the iDiv team built EBVCubeVisualizer, a QGIS plugin. It is like Panoply but tuned for EBV cubes: it shows the hierarchical structure and metadata while hiding the noisy low-level HDF5 attributes, and it lets you extract and visualise specific slices by **time, entity, scenario and metric** — each slice arriving as an ordinary QGIS raster layer you can then style or combine with other geodata.

Install it from the ZIP:

1. Download the plugin ZIP from the EBVCubeVisualizer repo (it is also on the QGIS plugin repository as “EBVCubeVisualizer”).
2. In QGIS: Plugins › Manage and Install Plugins › Install from ZIP, choose the file.
3. Tick “EBVCubeVisualizer” in the Installed tab, then open it from Plugins › EBVCubeVisualizer.

Use it when you want to *see* a dataset’s structure and slices interactively; use `ebvcube` in R when you want to compute, subset at scale, or produce reproducible figures.

5. The ebvcube package

This chapter is the reference you will keep coming back to. The package wraps the complexity of EBV netCDFs behind a small set of consistent verbs: **discover, read, subset, analyse, visualise** and **create**. Every example below uses data that ships with the package or downloads from the portal, so each is runnable as-is. The fuller, re-rendered versions live in the online docs.

5.1 Installing the package

Status note: `ebvcube` is currently on CRAN (version 0.5.3). — if `install.packages()` ever fails, fall back to GitHub or the r-universe below.

```
# Released version (recommended)
install.packages("ebvcube")

# Development version (newest features, may break)
# install.packages("devtools")
devtools::install_github("EBVcube/ebvcube", ref = "dev")

# b-cubed r-universe (daily binaries; pulls Bioconductor deps too)
install.packages("ebvcube", repos = c(
  "https://b-cubed-eu.r-universe.dev",
  "https://bioc.r-universe.dev",
  "https://cloud.r-project.org"))
```

The Bioconductor dependencies trip up clean installs

Three dependencies — `rhdf5`, `DelayedArray`, `HDF5Array` — are on Bioconductor, not CRAN, so a plain `install.packages("ebvcube")` cannot fetch them. If it complains one is missing:

```
install.packages("BiocManager")
BiocManager::install(c("rhdf5", "DelayedArray", "HDF5Array"))
```

On Linux you also need system libraries first: `libhdf5-dev libgdal-dev libproj-dev libgeos-dev libudunits2-dev`. Windows/macOS get these bundled with the CRAN binaries.

5.2 The API at a glance

Group	Function	Purpose
Discover	ebv_datacubepaths()	List all data cubes inside a file.
	ebv_properties()	Read the full metadata of a file or cube.
	ebv_download()	List / download datasets from the EBV Data Portal.
Read & subset	ebv_read()	Read a cube (or slice) into R.
	ebv_read_bb()	Read a spatial subset by bounding box.
	ebv_read_shp()	Read a spatial subset by shapefile.
	ebv_analyse()	Summary statistics for a cube or subset.
	ebv_resample()	Change the spatial resolution.
	ebv_write()	Write data back to disk as a GeoTIFF.
Visualise	ebv_map()	Quick classed map of one slice.
	ebv_trend()	Quick time-series plot (and returns the values).
Create	ebv_metadata()	Build the metadata JSON from R.
	ebv_create() / _taxonomy()	Create an empty EBV netCDF (optionally with taxonomy).
	ebv_add_data()	Fill the empty cubes with data.
	ebv_attribute()	Edit a single attribute after the fact.

5.3 The 30-second cycle: discover → map

A complete read-and-plot loop, the first thing to run against any new file:

```
library(ebvcube)
dir.create("data/ebv", recursive = TRUE, showWarnings = FALSE)

# Fetch the example dataset (id = 1, ~4 MB) from the portal
file  <- ebv_download(id = 1, outputdir = "data/ebv", overwrite = FALSE)

# Discover the cubes, then map the first one
cubes <- ebv_datacubepaths(file, verbose = FALSE)
ebv_map(file, cubes[1, 1], entity = 1, timestep = 2, classes = 9)
```

5.4 Discovering and reading

`ebv_properties()` without a path returns file-level metadata; with a path it adds the metric/scenario/cube slots. `ebv_read()` pulls a cube (or slice) into R, in the class you choose with `type`:

type	Returns
"r"	terra::SpatRaster (default — best for spatial work).
"a"	Base R array, read fully into memory.
"da"	Lazy, on-disk DelayedArray for cubes too big for RAM (see 7.2).

```
prop <- ebv_properties(file, verbose = FALSE)
prop@general$title          # what is this dataset?
prop@temporal$dates         # the time axis, as Date objects

dc  <- cubes[1, 1]          # a datacube path
arr <- ebv_read(file, dc, entity = 1, timestep = 1:3, type = "a")
dim(arr)
#> 180 360 3   (lat, lon, time — see the transpose note)
```

For subsets, `ebv_read_bb()` takes a bounding box `c(xmin, xmax, ymin, ymax)` (longitudes first), and `ebv_read_shp()` clips to a polygon. Both can stream straight to a GeoTIFF via `outputpath` instead of returning to memory. `ebv_analyse()` gives min/mean/quartiles/sd/n in one call and accepts the same `subset` argument.

```
# An African window out of the global file
bb  <- c(-26, 64, -47, 38)
sub <- ebv_read_bb(file, dc, entity = 1, timestep = 1, bb = bb)

# Summary stats for that region
ebv_analyse(file, dc, entity = 1, subset = bb, verbose = FALSE)[c("mean", "n")]
```

5.5 Common pitfalls

- **Hard-coding cube paths.** Scenario/metric names differ between datasets. Call `ebv_datacubepaths()` first, every time.
- **Name vs units mismatch.** A metric name ending in `(%)` may have units “Percentage points”. When labelling plots, trust `@metric$units`, not the name.
- **“Not enough RAM.”** The cube is bigger than the package thinks you can hold. Subset first (`ebv_read_bb/shp`), or read lazily with `type = "da"`. `ignore_RAM = TRUE` only switches off the check — it does not make the read lazy.
- **Map shifted by 180°.** The file uses a `[0, 360]` longitude convention; wrap it with `terra::rotate(r)`.

6. Visualisation

Two registers: the built-in functions for fast exploration, and `ggplot2` + `tidyterra` for publication figures. The figures in this chapter are from a synthetic Scandinavian-wildlife cube (Chapter 7), included as worked examples of the patterns.

6.1 Quick exploration: ebv_map() and ebv_trend()

`ebv_map()` draws a classed choropleth of a single slice; `ebv_trend()` plots a metric's trajectory over time **and** returns the underlying numeric values, so you can capture them for further use.

```
ebv_map(file, dc, entity = 1, timestep = 1, classes = 5, verbose = FALSE)

vals <- ebv_trend(file, dc, entity = 1, method = "mean", verbose = FALSE)
vals   # the plotted means, returned for further use
```

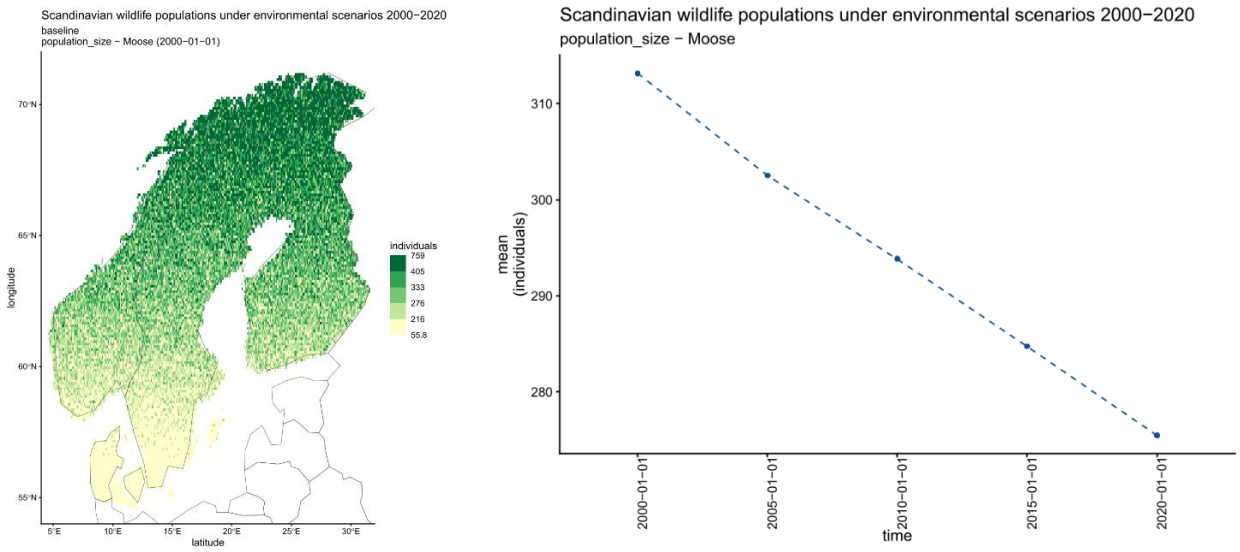

Figure. *Two one-line views of the Scandinavian-wildlife cube. Left: ebv_map() draws a single-slice choropleth — moose population, Baseline scenario, year 2000. Right: ebv_trend() plots the mean moose population size across 2000–2020.*

6.2 Pick entities by name, not number

Entity order can change when a dataset is updated, so select the name rather than a fixed index:

```
ents <- ebv_properties(file, verbose = FALSE)@general$entity_names
i_wolf <- which(grepl("wolf", ents, ignore.case = TRUE))
ebv_map(file, dc, entity = i_wolf, timestep = 1, classes = 5)
```

6.3 Custom maps with ggplot2 + tidyterra

Read the cube as a `SpatRaster` (`type = "r"`) and pipe it into `geom_spatraster()`. Layer country outlines with `geom_sf()` (for `sf` objects) or `geom_spatvector()` (for `terra` vectors) — the geom must match the object type. The cleanest global borders come from `rnaturalearth::ne_countries()`.

```
library(ggplot2); library(tidyterra); library(terra)
p2 <- ggplot(df_p2) +
  geom_tile(aes(lon, lat, fill = value)) +
  geom_sf(data = scandinavia_gadm, fill = NA, colour = "grey15", linewidth = 0.25) +
  scale_fill_viridis_c(option = "mako", direction = -1, name = "Individuals", na.value = NA) +
  coord_sf(xlim = c(4, 32), ylim = c(54, 72)) +
  facet_wrap(~ scenario, nrow = 1) +
  labs(
   title = "Wolf Population — Scenario Comparison",
   subtitle = "End-of-period snapshot (2020)  |  SYNTHETIC DATA",
   x = "Longitude",
   y = "Latitude",
   caption = common_caption) +
  theme_minimal(base_size = 10) +
  theme(
   plot.title = element_text(face = "bold", size = 14),
   plot.subtitle = element_text(colour = "firebrick", face = "italic"),
   plot.caption = element_text(colour = "grey40", size = 6, hjust = 0),
   strip.text = element_text(face = "bold", size = 11),
   legend.position = "right")
print(p2)
```

6.4 Small multiples: facet over time, entities or scenarios

Read several layers as a multi-layer `SpatRaster`, name the layers, and `facet_wrap(~ lyr)`. The same template compares time steps, entities, or scenarios — only the stacking variable changes.

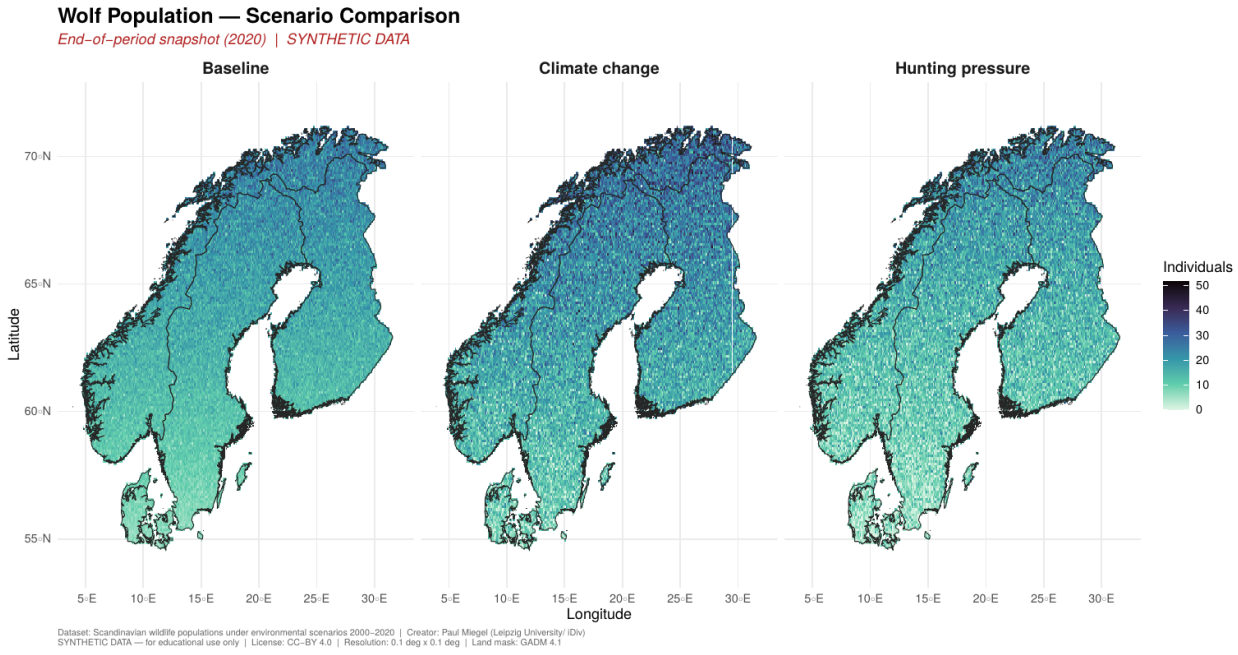

Figure. *Faceting in action: the wolf population under three scenarios (Baseline / Climate change / Hunting pressure), end-of-period snapshot, on a shared colour scale.*

Figure. *Trend lines per species and scenario, built from ebv_trend() values re-plotted in ggplot2 with a ±1 SD ribbon.*

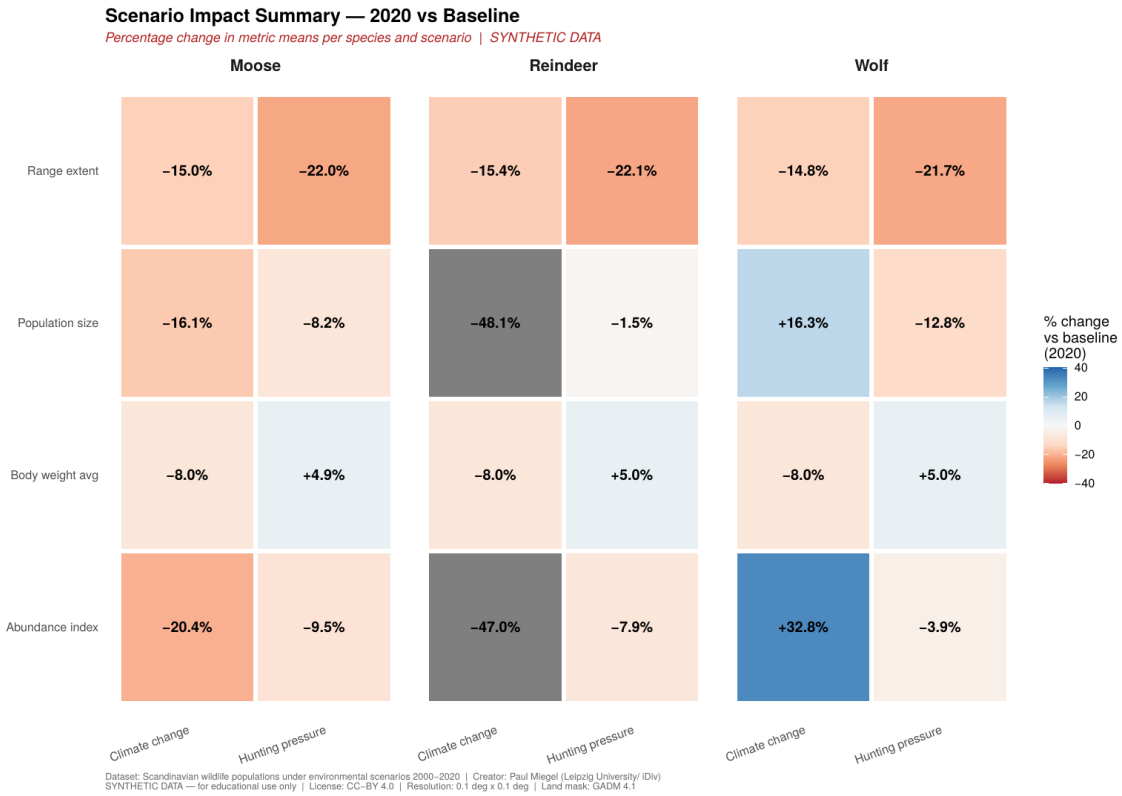

Figure. *A non-map summary: percentage change of each metric per species and scenario relative to baseline — the same data, reshaped into a heatmap.*

7. Creating your own EBV cube

This is the workflow for going from your own raster outputs to a portal-ready EBV netCDF — or simply a portable, well-described file. It is also the basis of the worked case study at the end of this chapter.

7.1 The four-step pipeline

1. **Metadata JSON** — either fill in the portal form and download the JSON (safest, because it enforces the controlled vocabularies and creates Portal ID), or build it from R with `ebv_metadata()`.
2. **Create the empty netCDF** with `ebv_create()`, which matches the JSON and writes the full structure filled only with the fill value.
3. **Add data** with `ebv_add_data()`, one entity (and optionally one time slice) at a time, from GeoTIFFs or in-memory rasters.
4. **Fix attributes** after the fact with `ebv_attribute()` — no need to rebuild the file.

```
# 2. create the empty file from the JSON
ebv_create(jsonpath = json, outputpath = new_nc,
           entities = c("forest birds", "non-forest birds", "all birds"),
           epsg = 4326, extent = c(-180, 180, -90, 90),
           resolution = c(1, 1), fillvalue = -3.4e+38, overwrite = TRUE)

# 3. fill metric_1, entity by entity
ebv_add_data(filepath_nc = new_nc, metric = 1, entity = 1,
             timestep = 1:3, data = "entity1.tif", band = 1:3)

# 4. correct an attribute in place
ebv_attribute(new_nc, attribute_name = "units", value = "Percentage",
              levelpath = "metric_1/ebv_cube")
```

> **Always sanity-check the empty file before adding data**
>
> Right after `ebv_create()`, confirm the structure with `ebv_properties(new_nc)@general$entity_names` and `ebv_datacubepaths(new_nc)`, and drop the `.nc` into Panoply for a visual check of the hierarchy.

7.2 Big files: lazy reads and taxonomy

Two advanced reading notes that matter once cubes outgrow memory:

- `type = "da"` returns a **DelayedArray** backed by the file on disk: it behaves like an array but only reads the chunks an operation actually touches. `ebv_write()` understands it and block-writes results without materialising the whole thing.
- **Taxonomy.** For species data, `ebv_create_taxonomy()` takes a CSV with one column per taxonomic rank and embeds a `taxonomy` variable that the portal and downstream tools can filter on. From there, `ebv_add_data()` works exactly as before.

7.3 Worked case study: the Scandinavian wildlife cube

A synthetic EBV cube, built from scratch, demonstrates the full creation pipeline end-to-end — a useful teaching artefact precisely because the data is fake and the focus is the workflow. It models:

Aspect	Value
Title	Scandinavian wildlife populations under environmental scenarios 2000–2020
EBV class / name	Species populations / Species distributions
Entities	3 mammals: Moose, Reindeer, Wolf
Metrics	4: population size, abundance index, range extent, body weight average
Scenarios	3: Baseline, Climate change, Hunting pressure
Time	5 steps, 2000–2020
Grid	0.1° × 0.1°, WGS84; land mask from GADM 4.1 (supersampled)
Region	Denmark, Sweden, Norway, Finland

The whole workflow shown in the GitHub docs runs the exact four steps above: write the JSON metadata, `ebv_create()` the empty structure, generate synthetic per-cell values, `ebv_add_data()` them in, verify, and finally plot. The figures in Chapter 6 are all outputs of this cube — maps, scenario facets, trend ribbons and the impact heatmap — so it doubles as a gallery of what a finished cube can produce.

8. FAQ and troubleshooting

The issues that come up most, with the shortest fixes that worked. The online docs' FAQ chapter is the living version — add to it when you find something new.

8.1 Installation

- **"there is no package called 'rhdf5'" (or DelayedArray / HDF5Array).** Bioconductor deps. `install.packages("BiocManager"); BiocManager::install(c("rhdf5","DelayedArray","HDF5Array"))`.
- **"package 'ebvcube' is not available for this version of R."** Your R is too old (floor is 4.2). Update R, or install an older tagged ebvcube via `devtools::install_github("EBVcube/ebvcube@v0.5.0")`.
- **"HDF5 library version mismatched."** Reinstall rhdf5 from a clean session: `remove.packages("rhdf5"); BiocManager::install("rhdf5", force = TRUE)`.

8.2 Reading and plotting

- **"Not enough RAM."** Subset first, or read with `type = "da"`.
- **ebv_read_shp() returns all NA.** Either a CRS mismatch (reproject the shapefile to the file's CRS) or no spatial overlap (check `@spatial$extent` vs `st_bbox()`).
- **Map shifted 180°.** `terra::rotate(r)`.
- **Plotting is slow.** `ebv_map()` rasterises at full resolution; `terra::aggregate(r, fact = 4, fun = "mean")` before plotting for exploration.
- **Reads slow on a network drive.** HDF5 makes many small reads; copy the file to local disk first.

8.3 Creating data

- **"entities length does not match metadata."** The `entities` vector and the count in the JSON disagree — align them.
- **ebv_add_data() dimension mismatch.** The GeoTIFF's resolution/extent/CRS does not match the netCDF; reproject/resample the TIFF to the target grid first.
- **Wrong units/titles after create.** Fix in place with `ebv_attribute()` — no rebuild needed.

8.4 Environment gotchas

- **Panoply won't launch.** Install Temurin JRE 17 (see 4.2).
- **Benign warnings while copy-pasting docs code.** A few chunks emit harmless warnings (missing optional attributes, GDAL UTF-8 notes, mapply length warnings); the rendered book suppresses them via `execute: warning: false`. They do not indicate a real failure.

Appendix A. Glossary

Term	Meaning
ACDD	Attribute Convention for Data Discovery (v1.3): global attributes for portal indexing.
CF Conventions	Climate and Forecast conventions (v1.8): coordinate/units encoding for netCDF.
datacube path	Slash-separated route to a cube, e.g. scenario_1/metric_2/ebv_cube.
DelayedArray	A lazy, on-disk array (Bioconductor) returned by ebv_read(type = "da").
EBV	Essential Biodiversity Variable — a standardised biodiversity measurement (GEOBON).
entity	The biological unit on a cube's 4th dimension: species, group, ecosystem type.
GEOBON	Group on Earth Observations Biodiversity Observation Network.
HDF5	Hierarchical data format underlying netCDF-4.
metric	An inner group: the quantity a cube measures (always present).
scenario	An optional outer group: e.g. SSP/RCP pathways.
SpatRaster	The raster class from the terra package.

Appendix B. Key links and resources

- EBV Data Portal — portal.geobon.org
- ebvcube package (GitHub)
- ebvcube on CRAN
- EBVCube-format definition
- EBVCubeVisualizer (QGIS plugin)
- Portal How-To (PDF)
- Panoply (NASA)
- What are EBVs? (GEO BON)

Appendix C. Known limitations and contributing

A few things to be aware of when installing, using, or contributing to the package:

- **Citation.** CRAN and GitHub citations differ slightly; the printed `citation("ebvcube")` can lag the DESCRIPTION by a release, so confirm it against the repository before citing. Cite individual EBV datasets by the DOI shown on their EBV Data Portal page.
- **Contributing.** Issues and pull requests are welcome on the GitHub repository